

Ask-AI System Architecture

Internal Technical Reference — v0.40.0

1. Overview

Ask-AI is a local-first CLI tool for interacting with Ollama LLM models, rewritten in Rust from the original Python ask-ai.py script. The system features a 5-layer chat architecture, tool calling with sandbox security, hybrid RAG retrieval (BM25 + sqlite-vec), and a feedback-driven memory lifecycle with Ebbinghaus decay. As of v0.40.0, the codebase comprises approximately 44K lines of Rust across 90+ source files.

2. Five-Layer Chat Architecture

The chat subsystem is organized into five dependency layers, where each layer only depends on layers below it. This strict layering prevents circular dependencies and enables future TUI migration without touching business logic.

Layer	Module	Purpose
L0	input.rs, view.rs	Traits — NO dependencies
L1	session.rs, cli.rs	Session + CLI argument parsing
L2	rustyline.rs, terminal.rs	Concrete input/view implementations
L3	repl_state.rs	Consolidated mutable state
L4	core.rs, command_handlers.rs	Business logic + tool loop
L5	repl.rs	Main loop coordinator

3. Context Overflow System

The system implements four context overflow thresholds that scale with the model's context window size. Each threshold triggers a specific action, from simple warnings to emergency result truncation:

Threshold	Usage	Action
PRE_TOOL	75%	Warning only
COMPACTION	88%	Auto-compact (summarize middle messages)
INTER_TOOL	94%	Warning during tool execution
EMERGENCY	97%	Truncate tool results

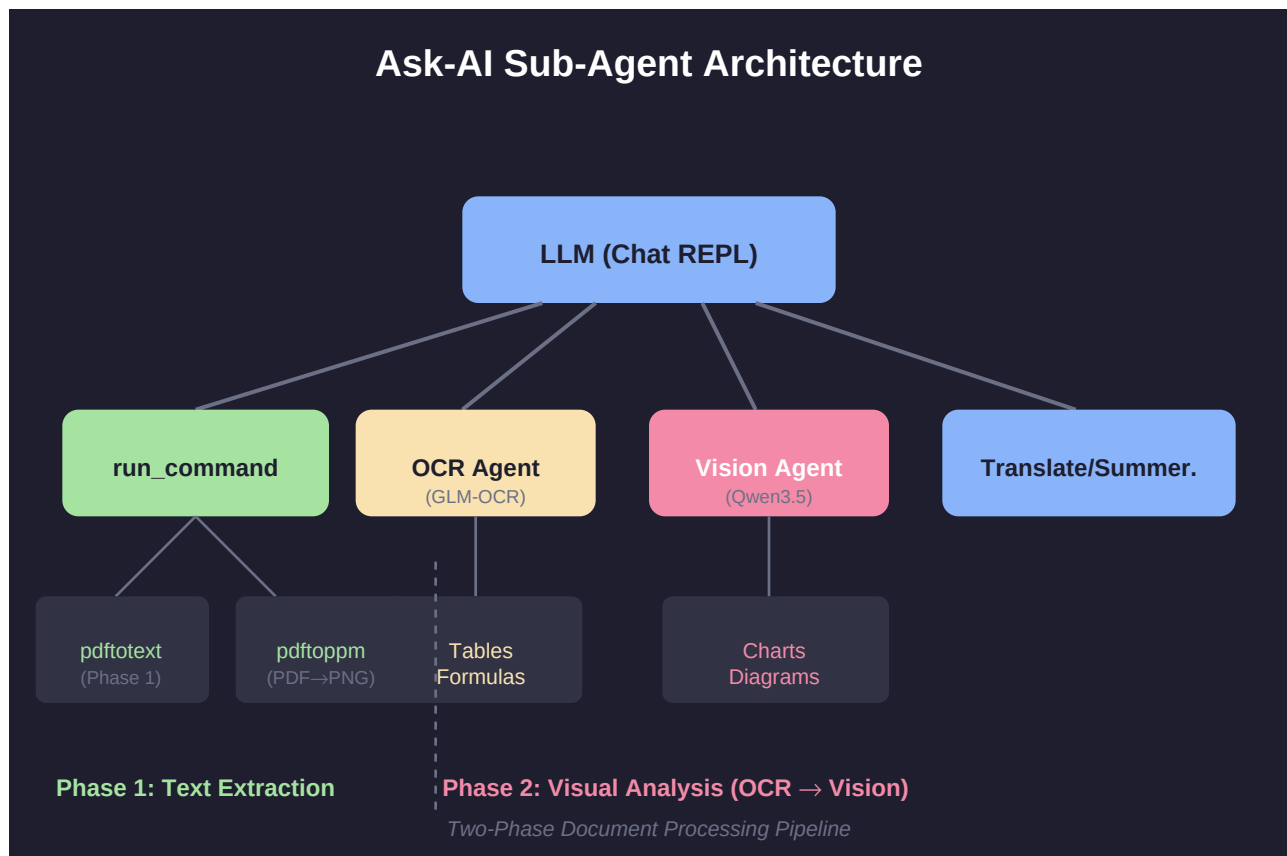
4. Fact Memory Lifecycle

Facts extracted from conversations follow an Ebbinghaus-inspired decay model. Each fact has a category-specific half-life (preferences: 180 days, facts: 30 days). Feedback from the user adjusts importance (+0.05 for good, -0.1 for bad), which in turn controls decay speed. Facts with importance ≥ 0.8 are never pruned. The contradiction detection pipeline uses 6 layers: exact match, normalized match, triple-based contradiction (exclusive vs accumulative predicates), semantic embedding, FTS5 keyword search, and startup verification.

Extraction is heuristic-only — no LLM involvement. The system classifies predicates as exclusive (e.g., 'prefers', 'works at', 'lives in') or accumulative (e.g., 'likes', 'enjoys'). Exclusive predicates with the same subject but different objects always produce contradictions. Accumulative predicates only contradict when object word overlap exceeds 0.3. Polarity flips (likes $\rightarrow$ hates) are always contradictions regardless of predicate type.

5. Sub-Agent Architecture

The sub-agent system enables the main LLM to delegate specialized tasks to purpose-built models. Each agent type is registered as an individual tool in the coordinator, with strict recursion prevention. The diagram below shows the two-phase document processing pipeline for PDF analysis.



In Phase 1, `pdftotext` extracts raw text from the PDF. If pages contain little or garbled text (indicating visual content), Phase 2 activates: `pdftoppm` converts those pages to PNG images, which are then sent to the OCR agent (for tables and formulas) or the Vision agent (for charts and diagrams). The strategy is to try OCR first, then escalate to Vision if the result is unsatisfying.

6. RAG Retrieval

The retrieval system uses a hybrid approach combining BM25 keyword search (weight 0.4) with sqlite-vec cosine similarity (weight 0.6), fused via Reciprocal Rank Fusion (RRF). Embeddings are generated locally via Ollama's embedding API. The `content_items` table unifies messages, notes, and documents under a single schema with type-specific decay half-lives: messages (90 days), notes (60 days), documents (120 days). A minimum of 5 messages must exist before retrieval activates, returning up to 5 relevant and 10 recent items.